

Question_3:

Code:

```
1  /*
2  Question No: 3
3  Given a directed graph, add the minimum number of edges so
4  that every city can reach every other city.
5  We first find strongly connected components.
6  */
7
8  #include <iostream>
9  #include <vector>
10 using namespace std;
11
12 vector<vector<int>> g,rg;
13 vector<int> vis,comp;
14 void dfs(int u){
15     vis[u]=1;
16     for(int v:g[u])
17         if(!vis[v]) dfs(v);
18     comp.push_back(u);
19 }
20 void dfs2(int u,int c){
21     vis[u]=1;
22     comp[u]=c;
23     for(int v:rg[u])
24         if(!vis[v]) dfs2(v,c);
25 }
26 int main(){
27     int n,m;
28     cin>>n>>m;
29     g.resize(n+1);
30     rg.resize(n+1);
31     vis.resize(n+1);
32
33     while(m--){
34         int a,b;
35         cin>>a>>b;
36         g[a].push_back(b);
37         rg[b].push_back(a);
38     }
```

```
39     for(int i=1;i<=n;i++)
40     |     if(!vis[i]) dfs(i);
41     fill(vis.begin(),vis.end(),0);
42     vector<int> order=comp;
43     comp.assign(n+1,0);
44     int c=0;
45     for(int i=n-1;i>=0;i--)
46     |     if(!vis[order[i]])
47     |         dfs2(order[i],++c);
48     if(c==1){
49     |     cout<<0;
50     |     return 0;
51     }
52     vector<int> in(c+1),out(c+1);
53     for(int u=1;u<=n;u++)
54     |     for(int v:g[u])
55     |         if(comp[u]!=comp[v]){
56     |             out[comp[u]]=1;
57     |             in[comp[v]]=1;
58     |         }
59
60     vector<int> from,to;
61     for(int i=1;i<=c;i++){
62     |     if(!in[i]) from.push_back(i);
63     |     if(!out[i]) to.push_back(i);
64     }
65     int k=max(from.size(),to.size());
66     cout<<k<<"\n";
67     for(int i=0;i<k;i++)
68     |     cout<<to[i%to.size()]<<" "<<from[(i+1)%from.size()]<<"\n";
69 }
```

Output:

Test Case_1:

```
4 5
1 2
2 3
3 1
1 4
3 4
1
2 1
PS C:\Users\Prabin Yadav\
```

Test Case_2:

```
4 5
1 3
2 1
1 2
2 4
1 4
2
2 1
3 1
PS C:\Users\Prabin Yadav\
```